Assessment Two

IT AND SOFTWARE OBJECT ORIENTED PROGRAMMING REPORT

Student Name: Gordon Zhao

Due Date: 19 June 2026

DESIGN OF CHARACTERS AND ENVIRONMENT

[Game name] is a first-person shooter/puzzle game set within the vast, shifting testing infrastructure of Hermes Inc

Characters

- The Player - An armed, highly mobile Hermes Inc. test subject attempting to break out of the facility. Never seen directly (first-person)
- Target dummies and defense system

Environment (including items)

- The Hermes Inc. Arenas(or something) - The primary setting. A massive, procedurally generated complex
- Jump Pads - Environmental hazard/mobility tools. Used to dramatically launch the player upward or forward to maintain aerial momentum during combat.
- Weapons / Guns - The player's primary tool for survival. Fires physical Rigidbody projectiles that deal damage to targets.
- Outer Bounds - Solid boundary walls and safety net floors automatically generated around each WFC room to contain the high-momentum combat.
- Other stuff

Define Characters as Objects

Identify Characteristics of Objects (Attributes) that can be Inherited from Parent Class

PlayerController	BasicEnemyAI	Projectile	Gun	Tiles	JumpPad
momentum	health (IDamageable)	linearVelocity	fireRate	weight	launchVelocity
isGrounded	movementSpeed	damage	recoilAmount	spawnRotation	isActive
isSlamming	stoppingDistance	owner	swayAmount	prefab	
movementSpeed	target		ammoCount	sockets[]	

Identify how characters (objects) will interact with each other and their environments through methods

PlayerController	BasicEnemyAI	Projectile	Gun	WFCGenerator	JumpPad
HandleMovement()	ChaseTarget()	OnTriggerEnter()	Fire()	CollapseCell()	OnTriggerEnter()
LaunchPlayer(Vector3)	Shoot()	DestorySelf()	ApplyRecoil()	ConstrainCell()	
HandleJump()	TakeDamage()			SpawnPrefabs()	
HandleSlide()	UpdateNavMesh()			GenerateArenaBounds()	

RESEARCH

The Genesis of Wumpus Research.

Hunt the Wumpus (1973) is a text-based adventure game written in BASIC by Gregory Yob. The player navigates a cave system represented as a network of rooms, avoiding hazards (pits, bats, and the Wumpus) and attempting to shoot the Wumpus with arrows. The game is entirely procedural, all logic is written as a sequence of instructions with no concept of objects or classes.

How procedural languages were used and compare 1 example to the OOP paradigm.

Describe the procedural language (e.g. Basic, COBOL, Fortran) and then compare it;s use in a table format - see https://en.wikipedia.org/wiki/Procedural_programming to start an investigation - remove this prompt before handing in.

Hunt the Wumpus Element	**OOP Translation**
Cave rooms (represented as numbered arrays)	Room class with attributes: roomID, connectedRooms[], hasHazard
The Wumpus (global variables tracking position/state)	Wumpus class with attributes: position, isAwake, and methods: move(), attack()
The Player (global position variable)	Player class with attributes: position, arrowCount, and methods: move(), shoot()
Bats (hazard triggered by room entry)	Bat class extending Hazard, with method: onPlayerEnter(player)
Arrow shooting (procedural if/else chain)	Arrow class with method: fire(direction), checking room connections
Game state (global win/lose flags)	GameManager class with methods: checkWinCondition(), checkLoseCondition()

SUCCESS CRITERIA

• Establish a set of quality success criterion for your project (5 minimum). Consider things like modularity, efficiency of code, replayability etc. Delete this comment prior to handing in your report

#	Success Criterion	How it will be measured
1	Modularity: code is organised into well-defined classes with single responsibilities	Class diagram reviewed, no class performs more than one major function
2	OOP principles: inheritance, encapsulation, and polymorphism are clearly demonstrated	Code reviewed for use of parent/child classes, private attributes, and method overriding
3	Puzzle functionality: all puzzles can be completed without bugs or soft-locks	Playtesting each puzzle from start to finish, documented in testing section
4	Player experience: the game communicates its mechanics clearly without a tutorial screen	User testing with classmates; feedback collected on clarity of controls and objectives
5	Performance: game runs at a stable frame rate with no significant lag	Frame rate monitored during playtesting; Unity Profiler used to identify bottlenecks
6	Code efficiency: no unnecessary loops, redundant variables, or repeated code blocks	Code reviewed and refactored; use of inheritance to eliminate duplication documented

SYSTEM DIAGRAMS

Data flow diagrams

Structure chart

Data dictionary

Class diagrams

TESTING AND EVALUATION

• Describe and document the application of methodologies to test and evaluate the code including:

Testing Technique	**Description of Technique**
Unit Testing	
Subsystem Testing	
System Testing	

Document how you completed these types of testing in your project **(used code snippets and describe what happened using your Journal as evidence)**

Unit

Subsystem

System

Testing Technique	**Description of Technique**
Black Testing	
White Testing	
Greybox Testing	

Document how you completed these types of testing in your project

Black

White

Greybox

Testing Technique	**Description of Technique**
Quality Assurance	
Alpha Testing	
Beta Testing	

Document how you completed these types of testing in your project

Quality Assurance

Alpha Testing

Beta Testing

Please note who in class tested your finalised product and the feedback given to you to implement.

Analyse and evaluate your solution against the quality success criteria.

Success Criteria	Analyse Solution	Evaluate Solution

Refine code for efficiency and optimisation

Document how these were achieved in your final product (use your Journal for evidence).

Efficient Measures Implemented

-
-
-
-
-

Optimisation Measures Implemented

-
-
-

BIBLIOGRAPHY